MLOps Fundamentals - Practice 3 Report

DVC experiments

Merlin Kägu

GitHub Repository: https://github.com/merlinkagu/MLOps_S26/tree/regression

1. Steps performed

1.1 Project Setup and GitHub Repository

The project was built on top of an existing NYC Green Taxi fare prediction pipeline from a previous practice session. The work was done on the regression branch of the MLOps_S26 repository. The project structure consisted of a src/ folder with pipeline components, a data/ folder tracked by DVC, and a models/ folder for saved model artifacts. Both data/ and models/ directories were added to .gitignore to prevent large files from being committed to Git directly.

1.2 Building the DVC Pipeline (dvc.yaml)

A dvc.yaml file was created to define the two-stage pipeline. The train stage takes both parquet data files as dependencies, reads parameters from params.yaml, trains a Random Forest Regressor, and outputs the saved model. The evaluation stage depends on the trained model and evaluates it on the full dataset. The pipeline was tested with dvc repro and visualized with dvc dag:

```
+----------------------------------------+        +----------------------------------------+
| data\green_tripdata_2021-01.parquet.dvc |        | data\green_tripdata_2021-02.parquet.dvc |
+----------------------------------------+        +----------------------------------------+
           ***                 *****                    ****                 ***
         ***                       ****            ****                       ***
      ***                              ***       ***                              ***
    ****                                +-------+                                  ****
        ********                        | train |                         ********
                ********                +-------+                   *******
                        *******                *                 *******
                               ********        *         ********
                                       ****    *     ****
                                          +----------+
                                          | evaluate |
                                          +----------+
+--------------------------------------------------+
| models\logistic_regression_classification.pkl.dvc |
+--------------------------------------------------+
```

1.3 Adding Configurable Parameters (params.yaml)

A params.yaml file was created to make the pipeline configurable without modifying source code. The file contains two sections: train (test_size, random_state, n_estimators) and data (train_path, test_path, outlier_distance). A load_params() function with proper logging and error handling was implemented in train.py:

```
def load_params(params_path: str) -> dict:
    try:
        with open(params_path, 'r') as file:
            params = yaml.safe_load(file)
        logger.debug('Parameters retrieved from %s', params_path)
        return params
    except FileNotFoundError:
        logger.error('File not found: %s', params_path)
        raise
    except yaml.YAMLError as e:
        logger.error('YAML error: %s', e)
        raise
    except Exception as e:
```

```
    logger.error('Unexpected error: %s', e)
    raise
```

1.4 Experiment Tracking with dvclive

The dvclive library was integrated into train.py to log metrics and parameters for each experiment run. I adjusted the code that was provided on the practice slides according to the classification regression task I chose. The following code block was added to the main() function:

```
with Live(save_dvc_exp=True) as live:
    live.log_metric('mae', mae)
    live.log_metric('rmse', rmse)
    live.log_metric('r2', r2)
    live.log_params(params)
```

Experiments were run using `dvc exp run` (instead of dvc repro) to enable dvclive experiment tracking. The command `dvc exp show` was used to compare experiments in a tabular format.

1.5 DVC Diff Commands

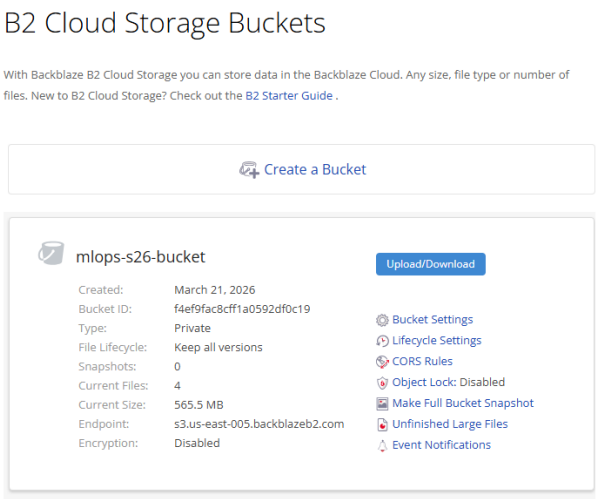
The following DVC commands were used to compare experiments and track changes:

- dvc params diff HEAD~1 — shows parameter changes between current workspace and previous commit
- dvc metrics diff — compares metric values across commits
- dvc plots diff — generates visual plots of metric evolution

Path	Param	HEAD~1	workspace
params.yaml	data.outlier_distance	-	150
params.yaml	data.test_path	-	data/green_tripdata_2021-02.parquet
params.yaml	data.train_path	-	data/green_tripdata_2021-01.parquet
params.yaml	train.n_estimators	-	100
params.yaml	train.random_state	-	42
params.yaml	train.test_size	-	0.2
dvclive\params.yaml	data.outlier_distance	-	150
dvclive\params.yaml	data.test_path	-	data/green_tripdata_2021-02.parquet
dvclive\params.yaml	data.train_path	-	data/green_tripdata_2021-01.parquet
dvclive\params.yaml	train.n_estimators	-	100
dvclive\params.yaml	train.random_state	-	42
dvclive\params.yaml	train.test_size	-	0.2

1.6 Remote Storage with Backblaze B2

A Backblaze B2 cloud storage bucket (mlops-s26-bucket) was created and configured as the DVC remote.



An application key with Read & Write access was generated. The remote was added to DVC using the S3-compatible API and credentials were stored locally using the --local flag to avoid committing secrets to Git. Finally, dvc push was used to upload all tracked files (4 files pushed successfully):

```
pip install dvc[s3]
dvc remote add -d dvstore s3://mlops-s26-bucket
dvc remote modify dvstore endpointurl https://<endpoint>
dvc remote modify --local dvstore access_key_id <KEY_ID>
dvc remote modify --local dvstore secret_access_key <APP_KEY>
dvc push
```

2. Key findings

2.1 DVC Pipelines Enable Reproducibility

The main insight from these two lectures is that DVC solves a core problem in ML development: reproducibility. By defining the pipeline in dvc.yaml with explicit dependencies and outputs, DVC can detect exactly which stages need to be re-run when something changes. If only the model hyperparameters change, only the train and evaluate

stages re-run — data ingestion is skipped. This saves significant time and ensures consistent results.

2.2 Separating Configuration from Code

Using params.yaml to store hyperparameters instead of hardcoding them in source files makes the pipeline much more flexible. Changing n_estimators from 10 to 100 no longer requires editing Python code — only the YAML file changes. DVC tracks these parameter changes and knows when to re-run stages, making the experimental process cleaner and more auditable.

2.3 Experiment Tracking Without a Separate Server

dvclive integrates directly with DVC to log metrics and parameters for each experiment run. Unlike MLflow or Weights & Biases which require a separate tracking server, dvclive stores results as simple files (JSON, TSV) in the repository. The dvc exp show command then provides a clean comparison table of all experiments. This is a lightweight but effective approach for small to medium scale ML projects.

2.4 Data Versioning and Remote Storage

Large data files and model artifacts should never be committed to Git. DVC solves this by storing only small .dvc pointer files in Git while pushing the actual large files to a remote storage (Backblaze B2). This keeps the Git repository lightweight while still allowing full reproducibility — anyone who clones the repo can run dvc pull to get the exact data and models.

3. Best experiment

Three experiments were conducted by varying the n_estimators parameter of the Random Forest Regressor. The table below summarizes all experiments:

Experiment	**n_estimators**	**MAE**	**RMSE**	**R^2**
licht-cyma	10	3.7603	6.2714	0.8148
optic-feme	50	3.6806	6.1256	0.8233
beamy-vela (best)	**100**	**3.6731**	**6.1318**	**0.8230**

The best experiment achieved MAE of 3.67 and R^2 of 0.823, meaning the model explains 82.3% of the variance in fare_amount. The evaluation stage (testing on the full Jan+Feb dataset) showed even better results with MAE: 2.33 and R^2: 0.923, indicating the model generalizes well. Increasing n_estimators beyond 50 did not improve performance, suggesting the Random Forest had already converged for this dataset size and feature set.